GRAMMAR-CONSTRAINED REFINEMENT OF SAFETY OPERATIONAL RULES USING LANGUAGE IN THE LOOP

(REPRODUCTION AND EXTENSION STUDY)

Abolfazl Akhavi

Spring 2026

INTRODUCTION

Motivation

Safety-critical systems rely on operational rules.

Examples:

- Autonomous Vehicles
- Medical Systems
- Industrial Automation

Challenges:

- Rules are manually designed
- Inconsistencies may exist
- Edge cases are often missed

Research Problem

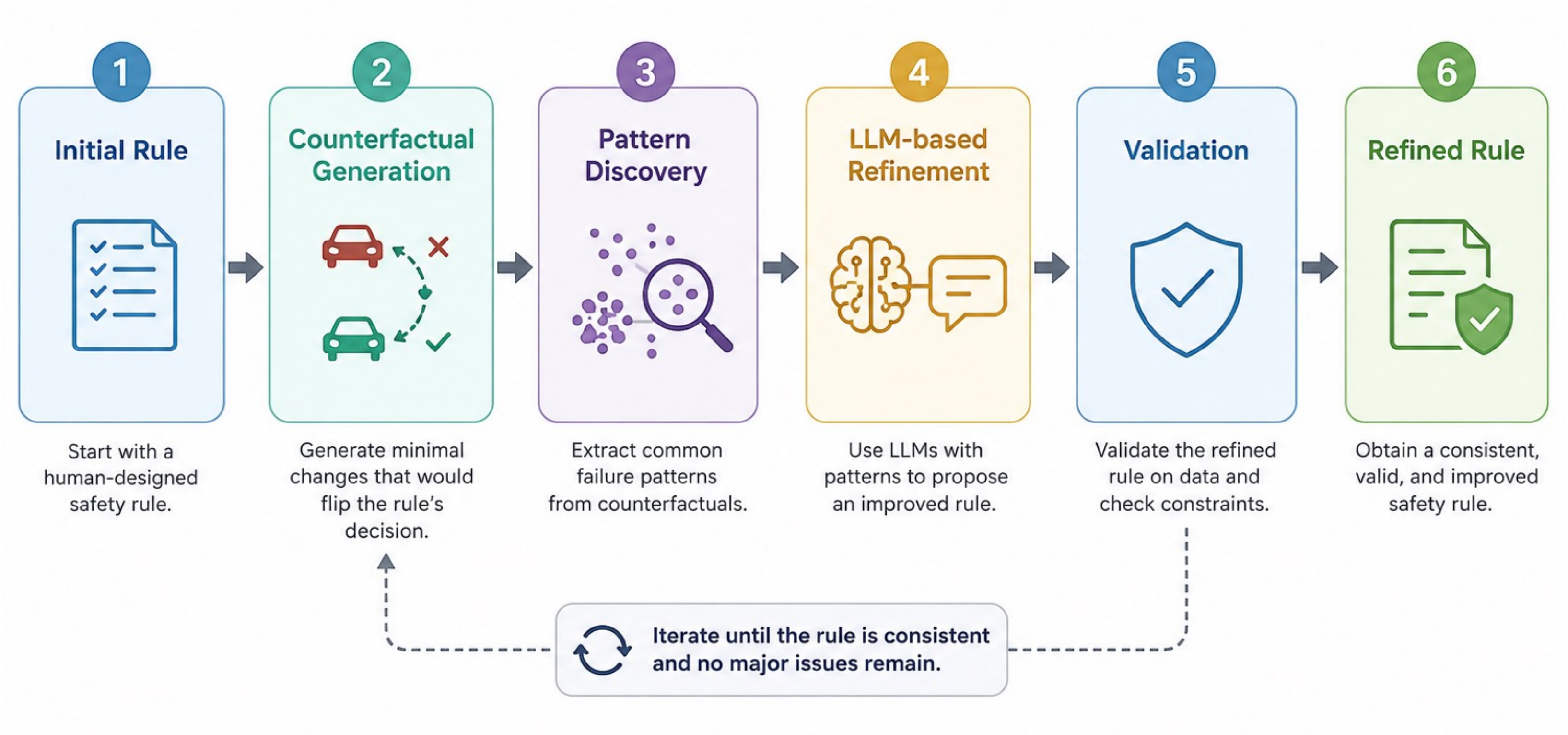
Can Large Language Models automatically refine safety rules?

Goal:

- Detect problematic rules
- Analyze failures
- Generate improved rules
- Preserve rule validity

Main Idea

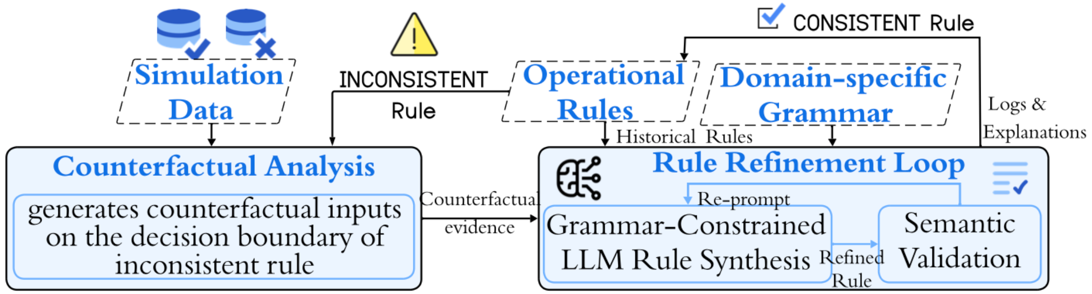
An iterative framework for refining safety operational rules using counterfactual analysis and LLM reasoning.



Paper Overview

The original paper proposes:

- Language-in-the-Loop framework
- Counterfactual analysis
- Grammar-constrained refinement
- Iterative rule improvement



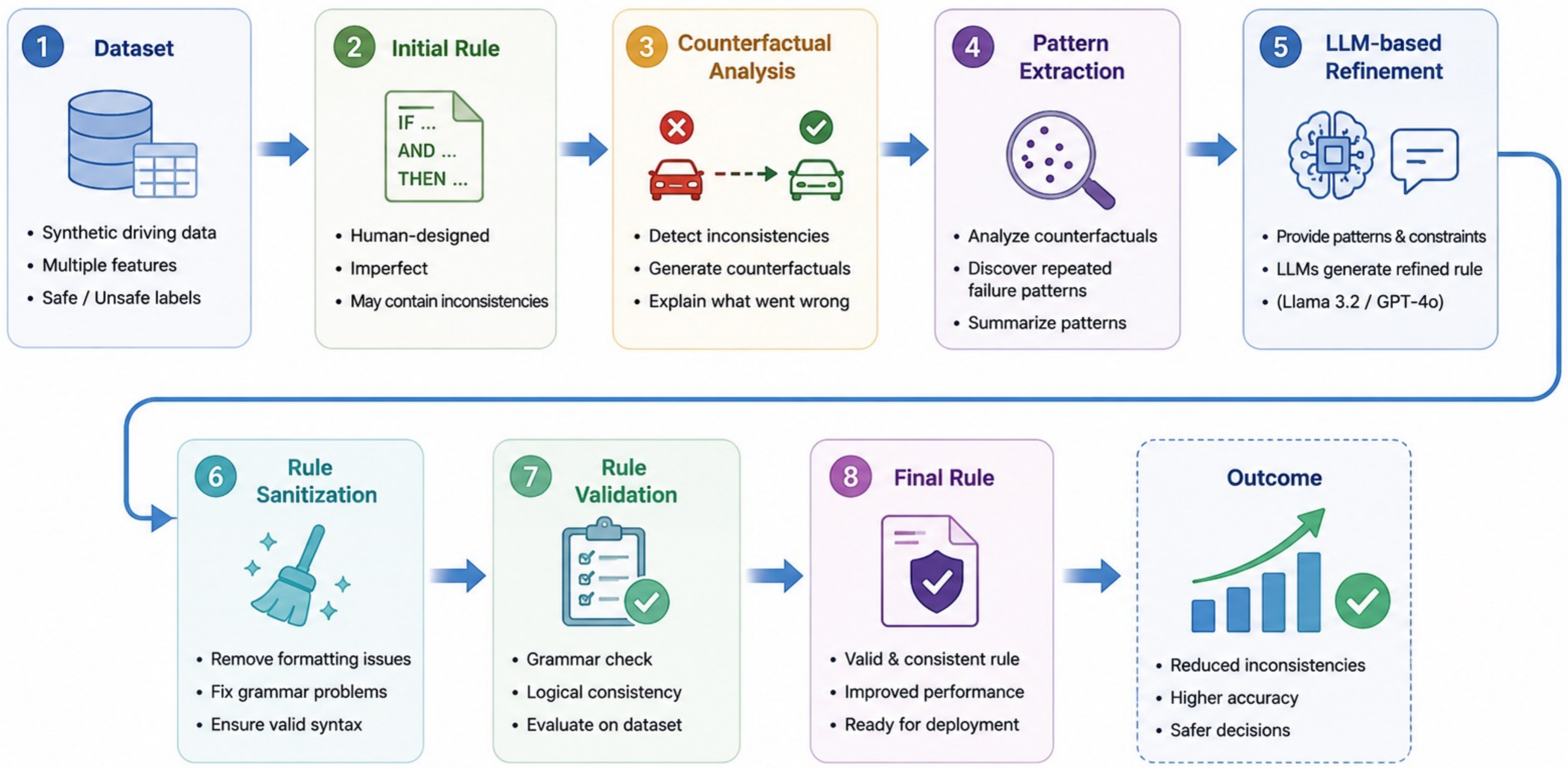
Our Contributions

Beyond reproduction:

- Pattern extraction from counterfactuals
- Rule Sanitizer module
- Rule Validator module
- GPT vs Llama comparison

IMPLEMENTATION

Experimental Pipeline



Dataset

Synthetic Autonomous Driving Dataset

Features:

- Speed
- Road Condition
- Visibility
- Pedestrian Presence
- Vehicle Distance

Output:

- Safe
- Unsafe

Initial Safety Rule

"If visibility is low and speed is high, then Unsafe"

Characteristics:

- Human-designed
- Imperfect
- Contains inconsistencies

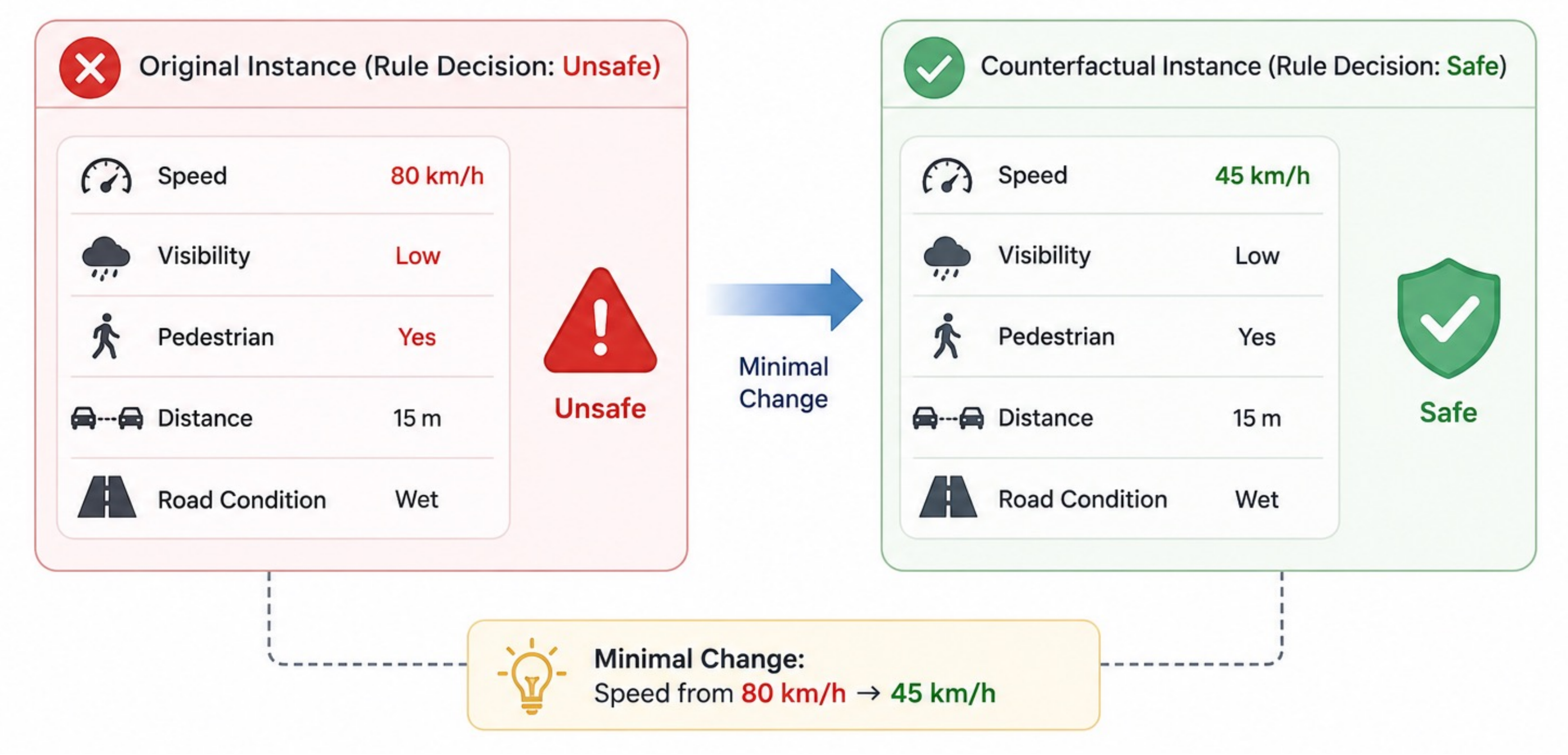
Detecting Inconsistencies

Objective:

- Find cases where
- Prediction $\neq$ Ground Truth
- These cases become candidates for refinement.

Counterfactual Generation

"What minimal change would reverse the decision?"



Why Counterfactuals?

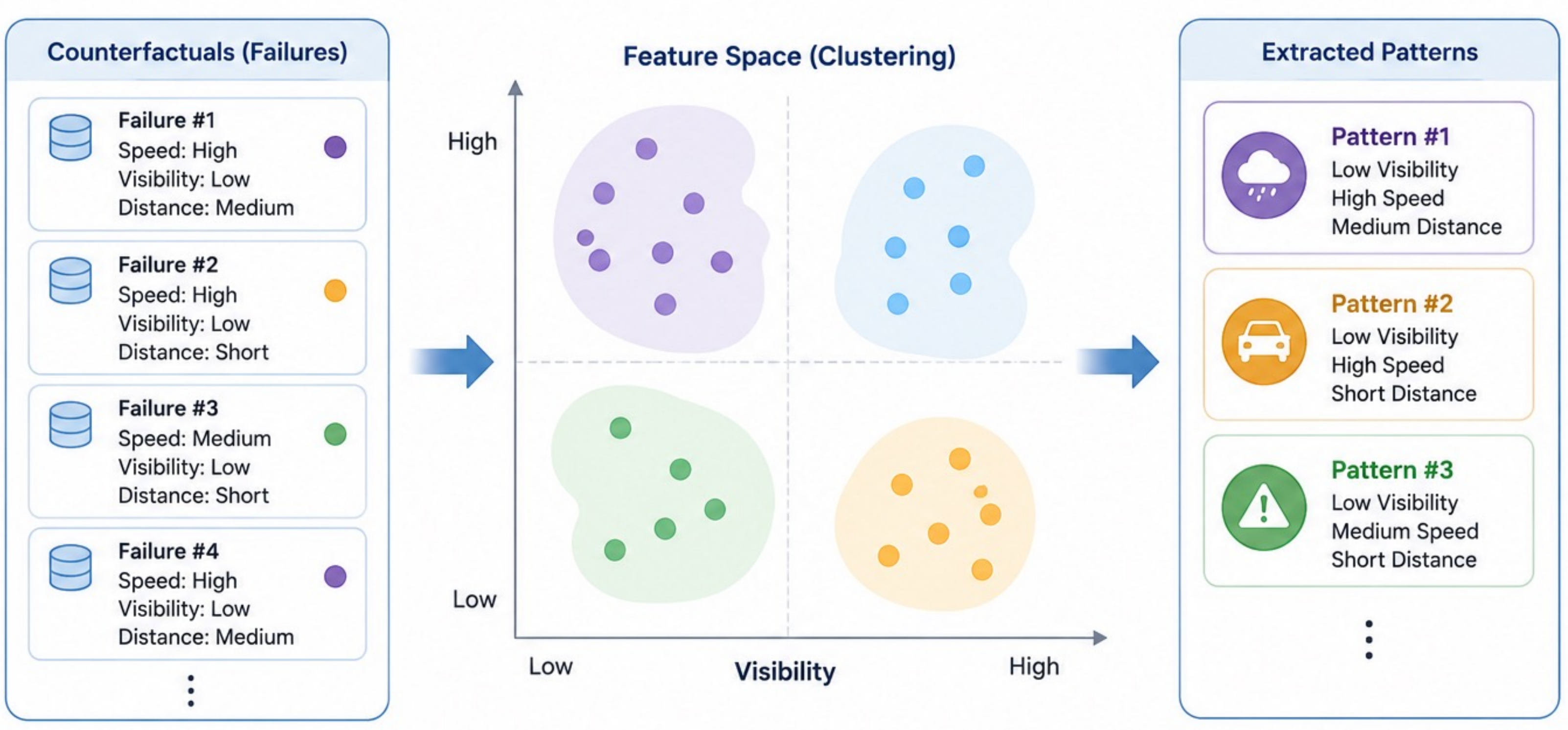
Counterfactuals reveal:

- Hidden weaknesses
- Boundary conditions
- Failure modes

They explain *what went wrong*.

Pattern Extraction

From multiple counterfactuals Repeated failure patterns are identified.



Pattern-Guided Refinement

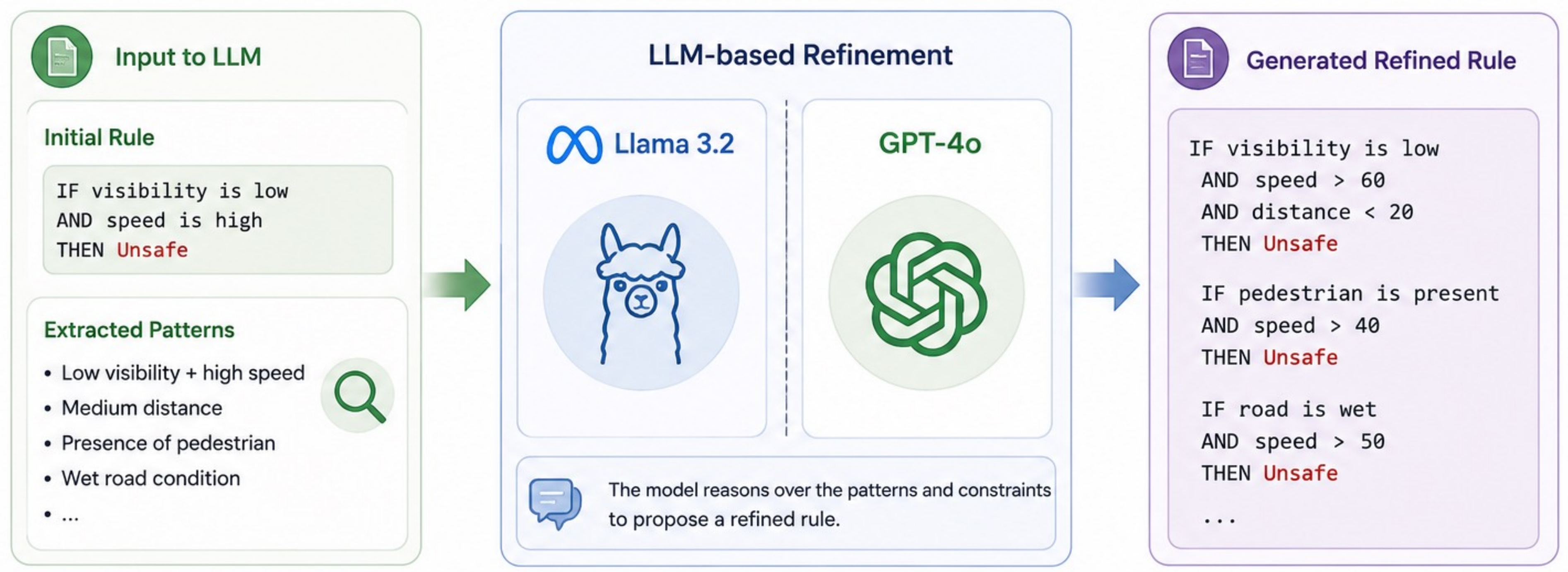
Extracted patterns are provided to the LLM.

Benefits:

- Better guidance
- More focused refinement
- Reduced hallucination

Rule Refinement with LLMs

Generate improved safety rules.



Rule Sanitizer

Purpose:

- Clean generated rules.

Checks:

- Grammar consistency
- Formatting
- Invalid tokens

Rule Sanitizer

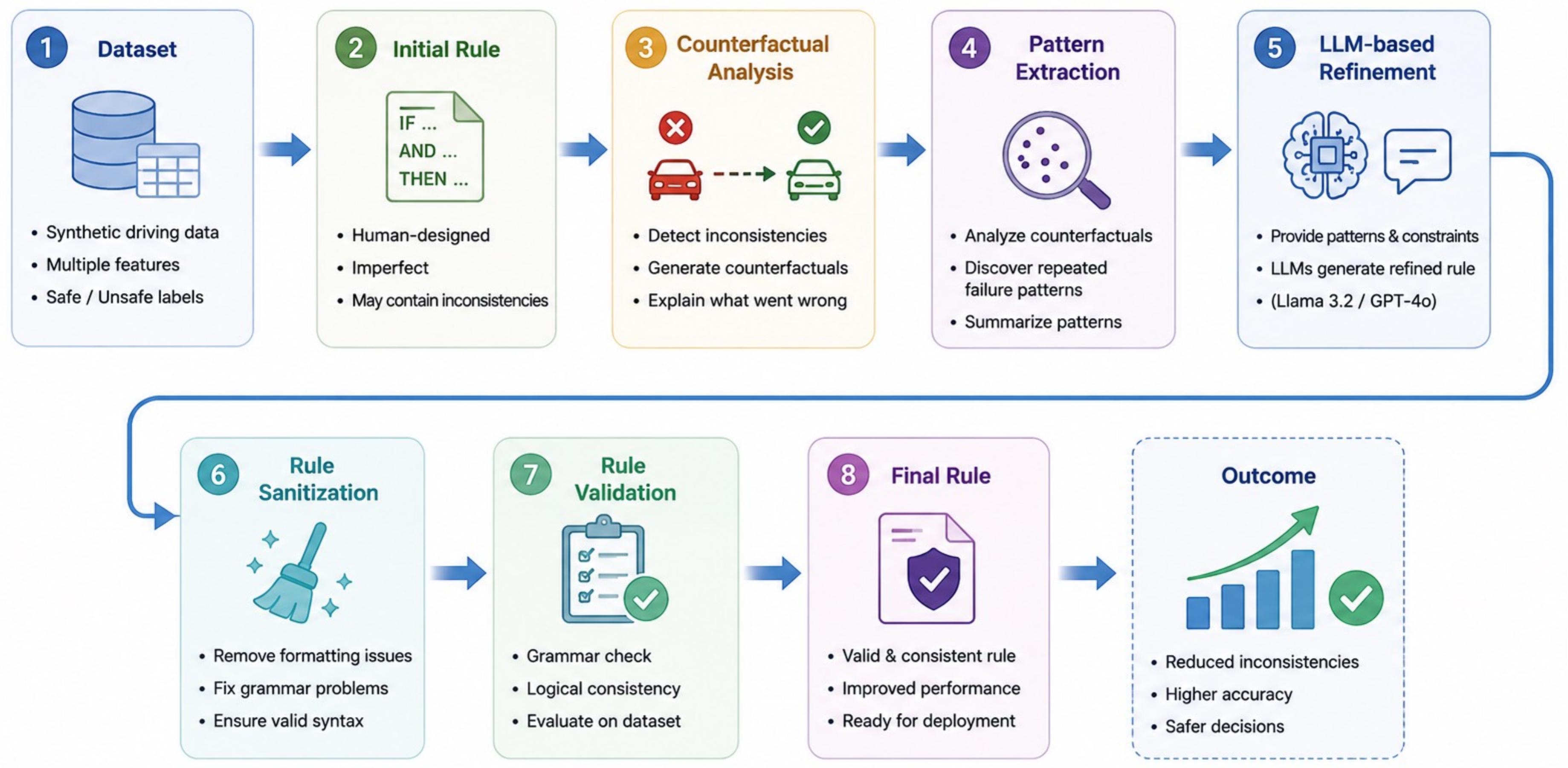
Purpose:

- Verify rule correctness.

Validation:

- Syntax constraints
- Logical consistency
- Dataset evaluation

Experimental Pipeline



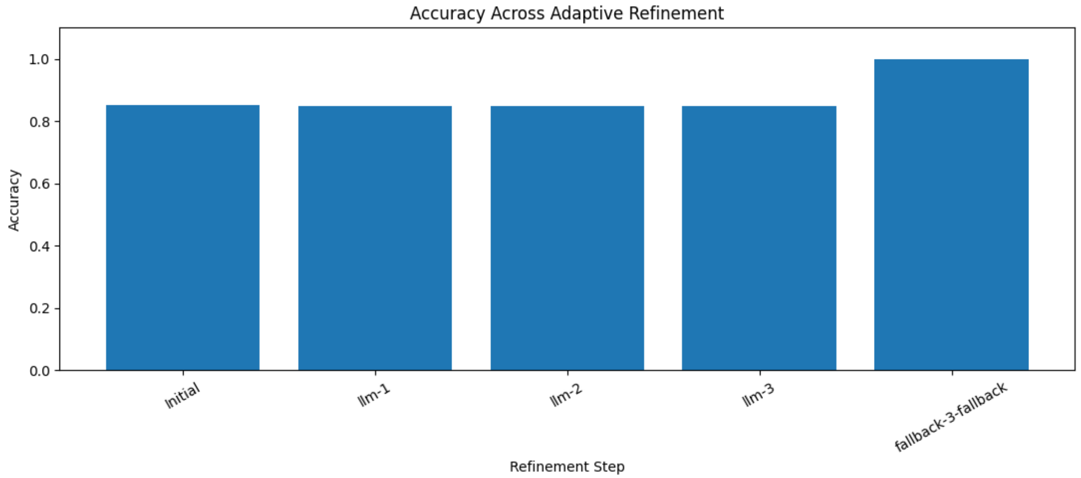
RESULTS

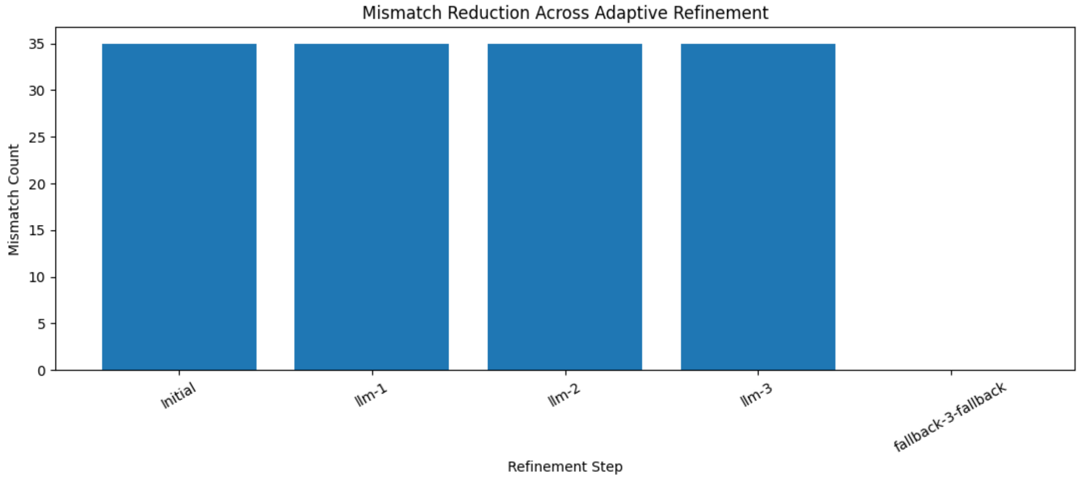
Llama 3.2 Results

Observed limitations:

- Unstable rule generation
- Lower consistency
- More invalid outputs

Required frequent correction.



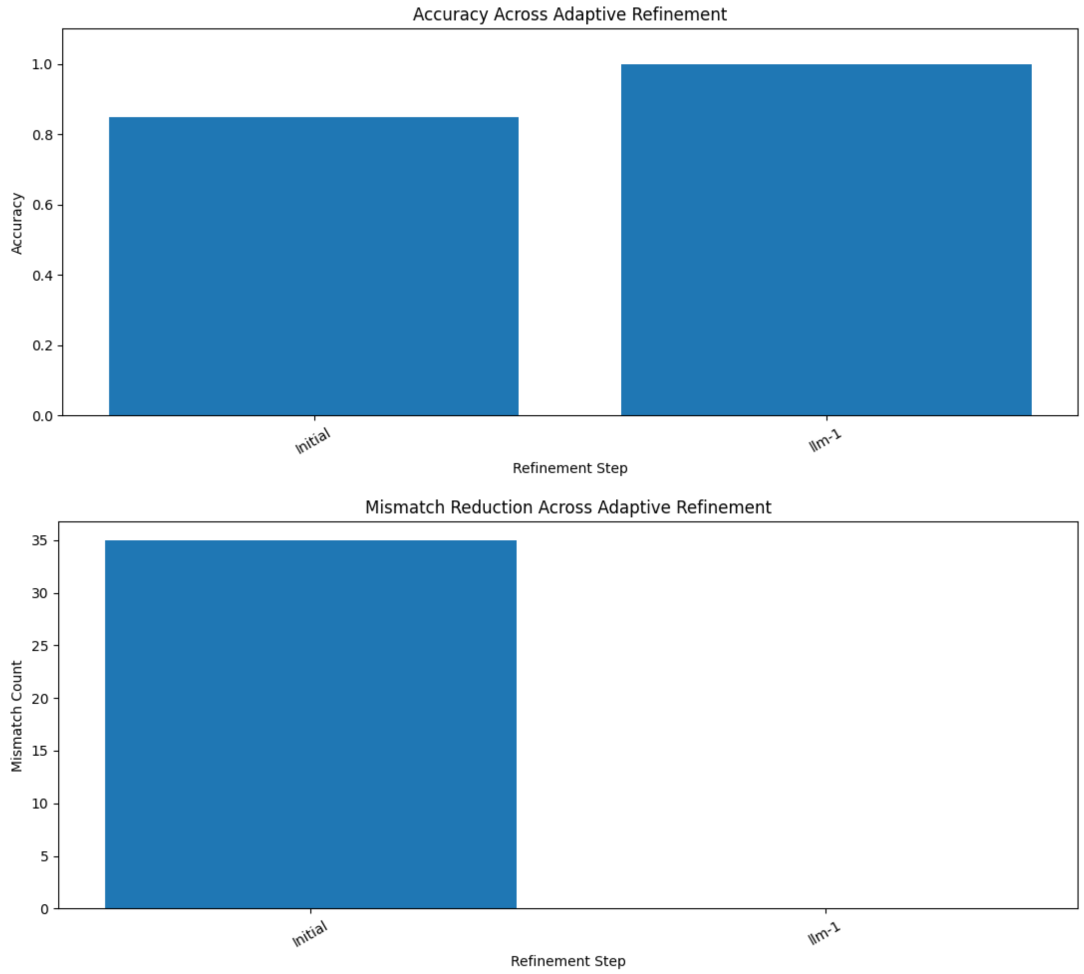


GPT-4o Results

Advantages:

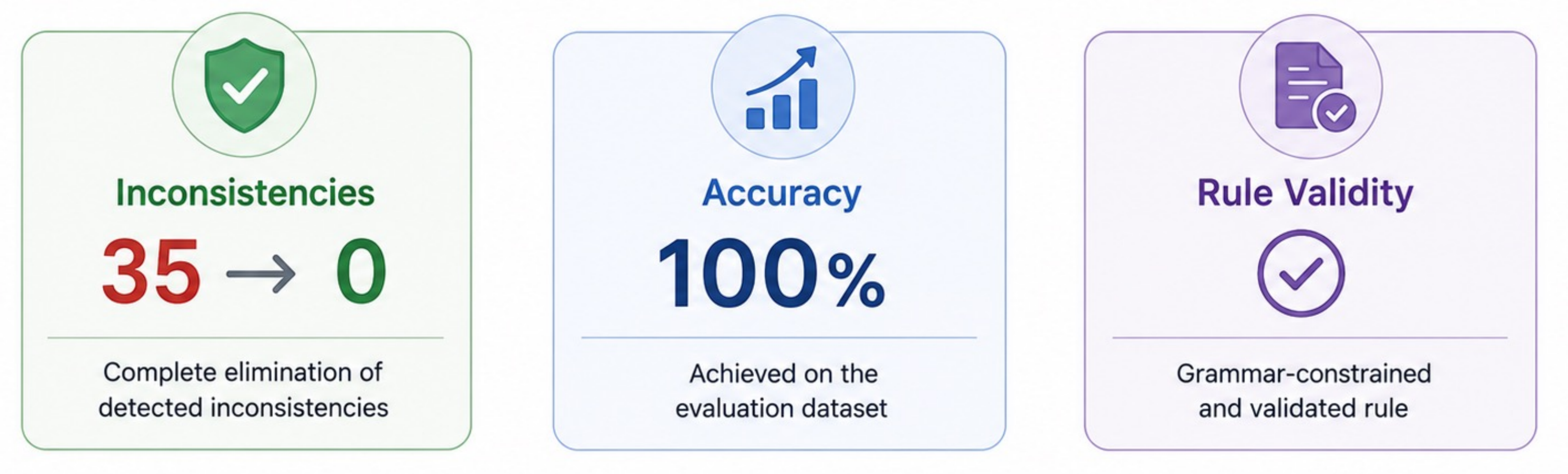
- Better reasoning
- More stable refinement
- Higher rule quality

Produced valid rules consistently.



Final Results

The refined rule eliminates inconsistencies and achieves perfect performance on the evaluation dataset.



CONCLUSION

Key Findings

- Counterfactuals provide valuable signals.
- Pattern extraction improves refinement.
- Validation modules increase reliability.
- GPT outperformed Llama in this task.

Conclusion

Grammar-constrained rule refinement can:

- Improve safety rules
- Reduce inconsistencies
- Leverage LLM reasoning effectively

Pattern-guided refinement further enhances performance.

THANK YOU